University Course Timetabling

METHODS, EVIDENCE AND BENCHMARKING: A REVIEW WITH CP-SAT AND SIMULATED ANNEALING

Every semester, universities must fit lectures, rooms, teachers, and student groups into a weekly timetable. Avoiding clashes is only the starting point. Rooms should be large enough, courses should be spread sensibly across the week, and student timetables should not be needlessly fragmented. We will review how current research tackles these decisions and compare two different approaches on the same public dataset.

RATIONALE AND RESEARCH QUESTIONS

The University Course Timetabling Problem (UCTP) is NP-hard. Abdipoor et al. [1] show that metaheuristics are widely used, but they are only one part of the field. Integer and constraint models remain relevant, often inside matheuristics [2,5]. Machine Learning is less common and usually helps with tasks such as choosing an algorithm, setting parameters, or estimating solution quality [2,3]. The review will cover all four strands instead of treating GA and SA as the whole state of the art.

The first question is whether the published evidence allows a reliable comparison of exact, metaheuristic, matheuristic, and learning-assisted methods. The second is experimental: given the same time limit, how do CP-SAT and Simulated Annealing (SA) differ in feasibility, timetable quality, stability, and running time? CP-SAT can report bounds and may prove optimality. SA cannot do that, but it can keep improving a feasible timetable with a comparatively simple search. The benchmark will show where that difference matters.

RESEARCH DESIGN

PHASE 1	Literature search. We will search Scopus, Web of Science, IEEE Xplore, ACM Digital Library, and ScienceDirect for peer-reviewed work published between 2015 and 2026. Backward citation searching will recover earlier papers that introduced benchmarks or methods still in use. The query will combine university course timetabling with terms for integer and constraint programming, metaheuristics, matheuristics, hyper-heuristics, Machine Learning, and benchmarks. A PRISMA-style diagram will record screening and exclusions.
PHASE 2	Analysis of the literature. For each paper, we will record the problem variant, dataset, constraints, objective, parameter tuning, stopping rule, baselines, statistical tests, and the availability of code and data. The papers will then be compared within four groups: exact methods, metaheuristics, matheuristics, and learning-assisted methods. A quality checklist will identify which results can be reproduced. For ML papers, we will state exactly what the model predicts or selects.
PHASE 3	Implementation. The CP-SAT model will encode every hard constraint and the official weighted objective. It will return the best timetable found together with a lower bound. SA will start from a timetable built by a greedy procedure, then improve it through move and swap operations, geometric cooling, and repair. The greedy construction will also be reported as a baseline. The three methods will share the parser, validator, scoring code, and output format.

EXPERIMENTAL PROTOCOL

The experiment will use the 21 public curriculum-based instances from ITC-2007 Track 3, based on data from the University of Udine [4]. The official score includes four soft penalties: room capacity, minimum working days, curriculum compactness, and room stability. Hard constraints must always be satisfied. We will tune the parameters on a reserved subset and then leave them unchanged. Final tests will use two fixed time limits. SA and any stochastic CP-SAT setting will be run with at least ten seeds per instance.

For each run, we will record whether the timetable is feasible, its official penalty, the time needed to find the first feasible timetable, and the time at which the best score appears. CP-SAT will also provide a bound. Results will be compared with published best-known scores. Confidence intervals, a Wilcoxon test, and an effect-size measure will support the comparison. Failed and timed-out runs will stay in the dataset.

EXPECTED CONTRIBUTION, FEASIBILITY, AND LIMITS

The paper will place the benchmark results beside the claims made in the literature. Code, seeds, parameter files, and raw results will be kept in the project repository. We will not train an ML model simply to add an AI component. Twenty-one benchmark instances do not support that choice. Instead, the review will examine the uses of ML that have actually been tested. ITC-2007 is public and anonymized, so the project does not process personal data. The final discussion will cover parameter sensitivity, random variation, hardware-dependent timing, and the limits of applying benchmark results to a live university.

REFERENCES

[1] S. Abdipoor, R. Yaakob, S. L. Goh, and S. Abdullah, 'Meta-heuristic approaches for the University Course Timetabling Problem,' Intelligent Systems with Applications, vol. 19, 200253, 2023. doi:10.1016/j.iswa.2023.200253.

[2] X. Gu et al., 'From Integer Programming to Machine Learning: A Technical Review on Solving University Timetabling Problems,' Computation, vol. 13, no. 1, 10, 2025. doi:10.3390/computation13010010.

[3] F. de la Rosa-Rivera et al., 'Algorithm selection for solving educational timetabling problems,' Expert Systems with Applications, vol. 174, 114694, 2021. doi:10.1016/j.eswa.2021.114694.

[4] L. Di Gaspero, B. McCollum, and A. Schaerf, 'The Second International Timetabling Competition (ITC-2007): Curriculum-based Course Timetabling (Track 3),' Technical Report QUB/IEEE/Tech/ITC2007/CurriculumCTT/v1.0, 2007.

[5] E. Rappos et al., 'A mixed-integer programming approach for solving university course timetabling problems,' Journal of Scheduling, vol. 25, pp. 391-404, 2022. doi:10.1007/s10951-021-00715-5.